

Understanding Our Advanced React Features

A team guide for understanding, presenting, testing, and safely simplifying the extra engineering in our codebase.

5	3	1
features explained	clear decisions	confident team

PERFORMANCE	ACCESSIBILITY	RELIABILITY
Lazy Dashboard	Focus and modal	Storage and CSS

01 / TEAM CODE GUIDE

Are these features too advanced?

They are not wrong. Some are core, some are useful quality improvements, and some are optional extras for a small course project.

SHORT ANSWER

Keep a feature when the team can **explain its purpose**, **find it in the code**, and **test its behavior**. Simplify it when the presentation risk is greater than the project value.

How to classify the current code

CORE	HELPFUL	OPTIONAL
Local Storage persistence and responsive styling are assignment requirements.	Defensive data checks and modal accessibility make the app safer and easier to use.	Lazy loading and automatic route focus go beyond the minimum specification.

The team rule

Keep the features we understand and can demonstrate. Simplify anything we would only memorize words for.

Quick decision order

Priority	Feature	Team decision
1	Defensive storage	Keep the reliable core; it supports a required feature.
2	Responsive CSS	Keep the design; organize it after teammate merges.
3	Modal focus	Prefer keeping if the team tests it with a keyboard.
4	Route focus	Optional; simplify the observer first if it feels unclear.
5	Lazy Dashboard	First candidate to remove; the performance gain is small here.

KEEP WHAT WE UNDERSTAND

SIMPLIFY OPENLY

TEST AFTER CHANGES



02 / TEAM CODE GUIDE

Lazy-loaded Dashboard

The Dashboard JavaScript is requested only when the user opens that route.

OPTIONAL EXTRA

DIFFICULTY: MEDIUM

VALUE: SMALL APP

In plain English

WHAT THE USER EXPERIENCES

The Kanban screen can load first. When Dashboard is opened, React fetches that page's code and briefly shows a loader if the download is not finished.

Where it appears

```
// src/App.jsx:1-15 and 61-68
const DashboardPage = lazy(
  () => import('./pages/DashboardPage.jsx')
)

<Suspense fallback={<RouteLoader />}>
  <Routes>{ /* routes */ }</Routes>
</Suspense>
```

What happens



Why it exists - and why it is optional

WHY KEEP IT

It demonstrates code splitting and can reduce the initial JavaScript download.

WHY SIMPLIFY IT

This app is small, so the performance difference may be hard to notice. It also requires **lazy**, **Suspense**, and a loader.

Safe simplification

```
import DashboardPage from './pages/DashboardPage.jsx'

// Then remove lazy, Suspense, and RouteLoader.
// Keep the /dashboard route exactly as it is.
```

15-SECOND PRESENTATION ANSWER

"The Dashboard uses React lazy loading. Its JavaScript loads when the Dashboard route opens, and Suspense gives temporary feedback. It is an optional performance improvement."

03 / TEAM CODE GUIDE

Keyboard focus management

React changes screens without a full page refresh, so the app helps keyboard and screen-reader users keep their place.

OPTIONAL ACCESSIBILITY

DIFFICULTY: ADVANCED

VALUE: HIGH FOR KEYBOARD

Two places where focus is managed

AFTER ROUTE CHANGE

src/App.jsx:17-50

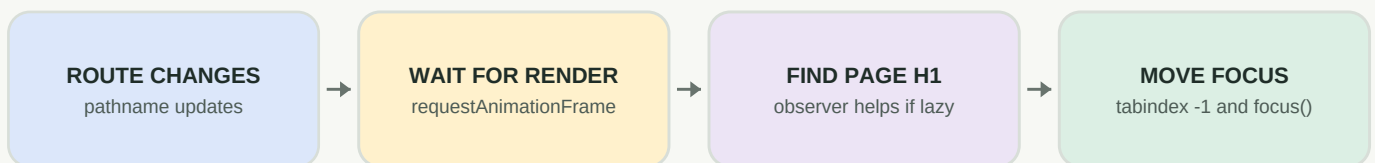
Focus moves to the new page's H1 and the page scrolls to the top.

AFTER TASK MOVEMENT

src/pages/KanbanPage.jsx:44-54, 83-88

Focus follows the task card into its new column and a live region announces the move.

Route focus flow



Three technical words to understand

Tool	Meaning in this project	Why it is present
<code>requestAnimationFrame</code>	Run after the browser starts the next paint.	Wait for React to update the page.
<code>MutationObserver</code>	Watch for new elements inside main content.	Wait for the lazy Dashboard H1.
<code>tabindex="-1"</code>	Allow code to focus an H1 without adding a Tab stop.	Announce the new page naturally.

Keep or simplify?

KEEP IF

The team can explain the route effect and test navigation using only Tab, Enter, and the move buttons.

SIMPLIFY IF

Remove **RouteFocusManager** first. Keep semantic buttons, the skip link, labels, and visible focus rings. If lazy loading is removed, a simple one-frame focus effect is enough.

15-SECOND PRESENTATION ANSWER

"Because React changes routes without refreshing the document, we move keyboard focus to the new page heading. We also return focus to a task after it moves, so users do not lose their position."

04 / TEAM CODE GUIDE

Accessible modal focus handling

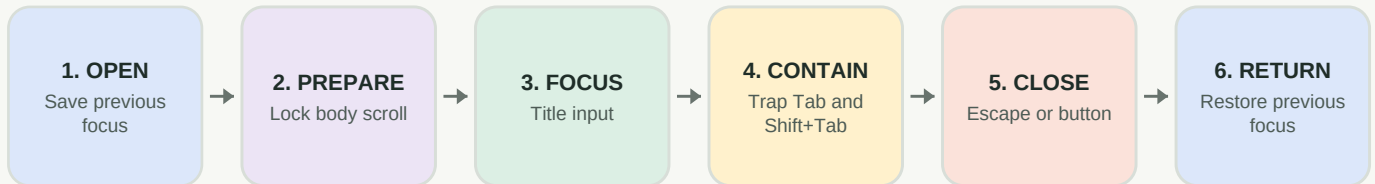
The create/edit task form behaves like a true keyboard-friendly dialog, not only a box drawn over the page.

HELPFUL EXTRA

DIFFICULTY: MEDIUM-ADVANCED

RECOMMEND: KEEP

What happens from open to close



Where it appears in our code

Location	Behavior
src/components/TaskForm.jsx:20-60	Refs, initial focus, Escape, Tab loop, scroll lock, cleanup, and focus restoration.
TaskForm.jsx:94-101	After validation, focus moves to the first invalid field.
TaskForm.jsx:120-140	role=dialog, aria-modal, labelled heading, and labelled close button.

Why it matters

WITHOUT THIS HANDLING

A keyboard user could Tab into controls hidden behind the modal, lose the field that needs correction, or close the form and have no idea where focus went.

Keep or simplify?

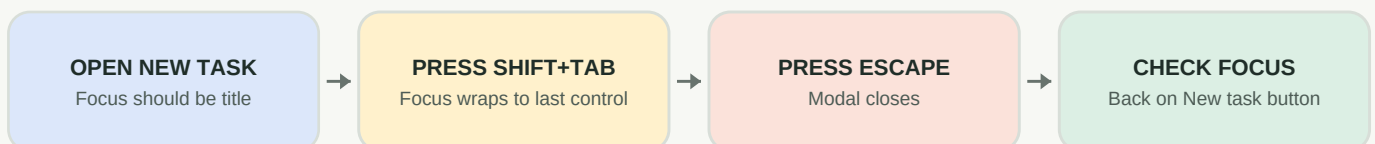
KEEP THE CURRENT VERSION

Recommended if the team tests forward Tab, Shift+Tab, Escape, validation errors, background scroll, and restored focus.

MINIMUM SIMPLER VERSION

Keep dialog ARIA, initial title focus, Escape, and focus restoration. The manual Tab trap and scroll lock may be removed, but accessibility becomes less complete.

Fast keyboard test



15-SECOND PRESENTATION ANSWER

"The task form keeps focus inside the dialog, supports Escape, prevents background scrolling, focuses errors, and returns users to the control that opened it."

05 / TEAM CODE GUIDE

Defensive Local Storage validation

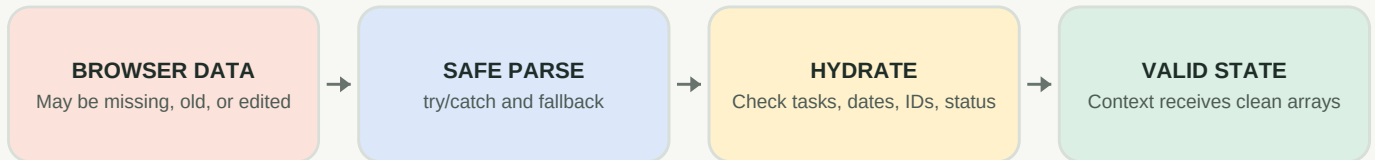
Local Storage is required. The advanced part is checking and repairing saved data before the app trusts it.

PERSISTENCE: REQUIRED

VALIDATION: HELPFUL

DIFFICULTY: MEDIUM

The problem and the protection



Where the work happens

File	Responsibility
src/utils/storage.js:1-21	Versioned keys plus safe JSON read/write with try/catch.
src/utils/taskHydration.js:3-109	Normalizes statuses, dates, IDs, fields, and category casing.
src/context/TaskContext.jsx:12-44	Loads once, hydrates, then saves tasks and categories after state changes.

Examples of data repair

Saved problem	What our app does
Broken JSON or wrong type	Use fallback data instead of crashing.
Unknown status or bad date	Use TODO or an empty date.
Duplicate or missing ID	Create a unique task ID.
Completion date on unfinished task	Clear it unless the task is DONE.
Category duplicates	Trim and merge names case-insensitively.

Minimum robust version if we simplify

```
try {
  const value = JSON.parse(localStorage.getItem(key))
  return Array.isArray(value) ? value : fallback
} catch { return fallback }
// Also keep valid status checks and default fields.
```

RECOMMENDATION

Keep the important checks. This code protects a required feature and has direct unit tests in `tests/taskHydration.test.js`.

OPTIONAL TO REMOVE

Duplicate-ID repair, strict calendar-date checking, and category-case normalization are the first candidates if the team needs less code.

15-SECOND PRESENTATION ANSWER

"Local Storage can contain malformed or old data. We safely parse it, normalize fields, and fall back instead of letting bad browser data crash the board."

06 / TEAM CODE GUIDE

Extensive custom CSS

The ideas are normal web design. The advanced-looking part is the volume: one stylesheet currently contains about 1,934 lines.

RESPONSIVE UI: REQUIRED

VOLUME: HIGH

RECOMMEND: ORGANIZE

What the stylesheet contains

Section	Approximate lines	Purpose
Tokens and accessibility	1-119	Colors, type, focus rings, screen-reader helpers, skip link.
Dashboard system	377-828	Hero, summary cards, charts, legends, responsive data views.
Kanban system	829-1354	Toolbar, columns, task cards, movement controls, empty states.
Task modal	1355-1551	Backdrop, dialog, form fields, validation, buttons.
Responsive and motion	1552-1934	Four breakpoints and reduced-motion support.

What is normal vs. what increases maintenance

GOOD FOUNDATIONS

CSS variables, Grid, Flexbox, responsive breakpoints, visible focus, clear contrast, and reduced motion all support the assignment.

OPTIONAL DECORATION

Gradients, blur, pseudo-elements, animation, color-mix, and many one-off selectors can be removed without changing task logic.

Recommended correction - after teammate merges

```
src/styles/
base.css      # tokens and global rules
navigation.css # navbar and shared buttons
dashboard.css # cards and charts
kanban.css    # columns and task cards
modal.css     # task form
responsive.css # breakpoints and reduced motion
```

LOWER-RISK OPTION

Keep one file and add stronger section comments. Splitting CSS before three branches merge can create avoidable conflicts and change cascade order.

Do not remove these just to reduce line count

KEEP THE FOUNDATIONS

Responsive breakpoints, visible **:focus-visible** rings, shared CSS variables, readable contrast, and reduced-motion support.

15-SECOND PRESENTATION ANSWER

"We used custom CSS because the assignment allows it and the two layouts need responsive control. Shared variables keep the design consistent; after integration, the large file can be organized by feature."

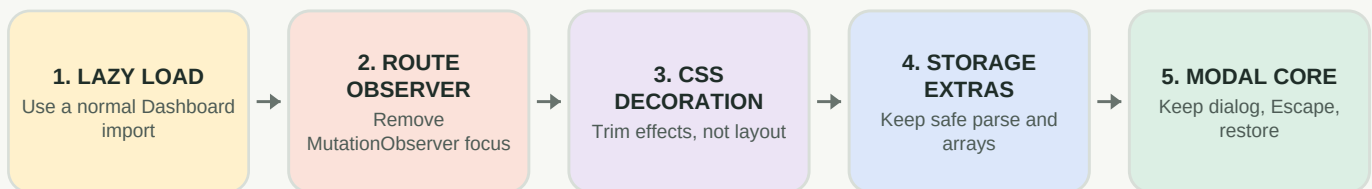
07 / TEAM CODE GUIDE

Keep, simplify, or refactor?

This matrix is the practical decision page for the whole team.

Feature	Required?	Difficulty	Main value	Recommendation
Lazy Dashboard	No	Medium	Loading performance	Optional - simplify first if needed.
Route and card focus	No	Advanced	Keyboard accessibility	Keep only if understood and tested.
Modal focus handling	No	Medium-advanced	Accessible task editing	Prefer keeping; test with keyboard.
Defensive storage	Partly	Medium	Stability and recovery	Keep the important checks.
Extensive CSS	Responsive style is required	Medium maintenance	Clear, eye-cooling UI	Keep design; organize later.

If the team wants a simpler codebase, use this order



Never remove assignment behavior

DATA	PERSISTENCE	DASHBOARD	DELIVERY
Create, edit, delete, move, categories, responsible person, dates, complete date.	Use the required Local Storage keys and restore after refresh.	Five cards and all three meaningful charts must use real task data.	Responsive pages, GitHub Pages, screenshots, member names, and usage README.

Overall verdict

POLISHED, NOT INVALID

The app is more engineered than the instructor's small teaching example, but the required project is also much larger. Do not claim every extra was required. Describe them honestly as deliberate quality improvements.

UNDERSTAND	TEST	EXPLAIN	SIMPLIFY IF NEEDED
------------	------	---------	--------------------

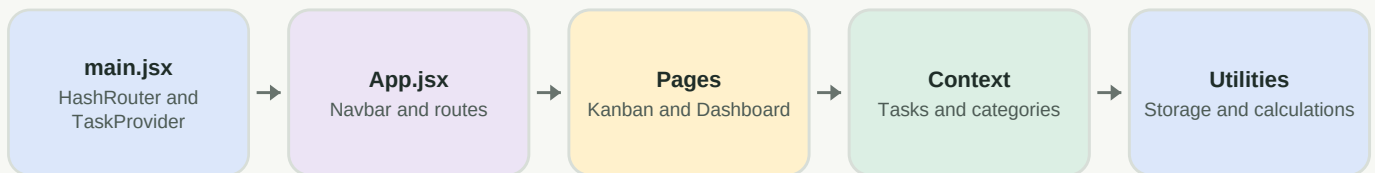
08 / TEAM CODE GUIDE

Likely instructor questions

Use these answers as concepts, not as a memorized script. Every member should be able to point to the related file.

Question	Clear answer
Why use HashRouter?	GitHub Pages is a static host. Hash routes work without server-side route rewriting. See <code>src/main.jsx</code> .
Why lazy-load only Dashboard?	It is a separate route and is not needed when Kanban opens first. See <code>src/App.jsx</code> .
Why <code>tabindex="-1"</code> on the H1?	It lets code focus the heading after navigation without adding it to the normal Tab order.
Why <code>MutationObserver</code> ?	The lazy Dashboard H1 may not exist immediately, so the observer waits for it.
Why restore modal focus?	Keyboard users return to the control where they started instead of losing their position.
Why try/catch around storage?	JSON parsing or browser storage can fail; fallback data keeps the app usable.
What if saved tasks are invalid?	Hydration checks the array and repairs or defaults individual task fields.
How is overdue calculated?	The due date has passed and status is not DONE.
How does Dashboard get task data?	<code>TaskProvider</code> owns state, <code>useTasks</code> exposes it, and calculation functions produce cards and chart counts.

Application structure in one picture



One sentence for each extra

Extra	Team explanation
Lazy loading	Load Dashboard code when its route is opened.
Focus management	Keep keyboard users oriented after screen or task changes.
Modal handling	Contain focus, support Escape, and restore the opener.
Storage validation	Turn browser data into safe application state.
Custom CSS	Provide consistent, responsive layouts for both pages.



09 / TEAM CODE GUIDE

Team readiness checklist

A feature is ready for submission only when the team can demonstrate it and explain the data flow around it.

Every member should be able to

- | | |
|--|--|
| <ol style="list-style-type: none">1. Trace data from Local Storage to TaskContext, pages, cards, and charts.2. Explain why moving to DONE sets today's completion date.3. Explain why moving out of DONE clears the completion date.4. State the exact overdue rule: due date passed and status is not DONE.5. Create, edit, delete, and move a task without help. | <ol style="list-style-type: none">6. Add a category, refresh, and confirm tasks and categories remain.7. Navigate the board, modal, and Dashboard using only the keyboard.8. Explain at least one advanced feature in their own words.9. Identify which extras are optional and which behavior is required. |
|--|--|

After any simplification or merge

```
npm test
npm run lint
npm run build
npm run dev

# Then test CRUD, movement, refresh, charts, mobile, and keyboard.
```

README reminders for final delivery

MUST INCLUDE

Project description, application screenshots, member names only, and basic usage instructions.

ADD AFTER MERGING

Repository URL, GitHub Pages URL, final screenshots, and instructions that match the finished interface.

Final team message

CONFIDENCE COMES FROM UNDERSTANDING

Advanced code is acceptable when it has a clear purpose, works reliably, and the team understands it. If a feature cannot be explained or safely changed, simplify it before submission instead of memorizing technical words.

TRACE THE FLOW

TEST TOGETHER

PRESENT HONESTLY